CHAPITRE 5
ALGÈBRE RELATIONNELLE

Par Robert DIASSÉ

A. INTRODUCTION

L'algèbre relationnelle constitue le **fondement théorique** du modèle relationnel et des systèmes de gestion de bases de données. Développée par Edgar F. Codd en 1970, elle fournit un ensemble d'opérations mathématiques permettant de manipuler les relations de manière formelle et rigoureuse.

Contrairement au SQL qui est un langage **déclaratif** (on décrit *ce que l'on veut* obtenir), l'algèbre relationnelle est un langage **procédural** (on décrit *comment* obtenir le résultat par une séquence d'opérations).

Place dans la démarche de modélisation

MCD (Conceptuel)	→ Que modéliser ?
↓	
MLD (Logique)	→ Comment structurer ?
↓	
Algèbre Relationnelle	→ Comment manipuler ?
↓	
SQL (Implémentation)	→ Comment implémenter ?

Objectifs du chapitre :

- Maîtriser les opérations fondamentales sur les relations
- Comprendre la composition d'opérations pour résoudre des requêtes complexes
- Établir le lien entre algèbre relationnelle et langage SQL

B. CONCEPTS FONDAMENTAUX

I. Définition d'une relation

En algèbre relationnelle, une **relation** est un ensemble de tuples (lignes) partageant les mêmes attributs (colonnes). Elle correspond à la notion de **table** dans le modèle logique.

Propriétés fondamentales d'une relation :

- Chaque tuple est **unique** (pas de doublons)
- L'**ordre des tuples** n'a pas d'importance
- L'**ordre des attributs** n'a pas d'importance
- Chaque attribut possède un **domaine de valeurs** défini

II. Propriété de fermeture

L'algèbre relationnelle est un système **fermé** :

- L'entrée d'une opération est une ou plusieurs relations
- La **sortie** est toujours une **relation**

Cette propriété permet d'**imbriquer** les opérations : le résultat d'une opération peut servir d'entrée à une autre opération.

III. Notations

- **Relation :** R, S, T
- **Attributs :** A, B, C
- **Schéma de relation :** $R(A_1, A_2, \ldots, A_n)$

— **Cardinalité :** nombre de tuples dans une relation, noté $|R|$

— **Degré :** nombre d'attributs d'une relation

IV. Relations d'exemple

Pour illustrer les opérations, nous utiliserons les relations suivantes :
Relation ÉTUDIANT

id_etudiant	nom	prenom	ville
1001	Dupont	Marie	Paris
1002	Martin	Pierre	Lyon
1003	Durand	Sophie	Paris
1004	Bernard	Jean	Marseille

Relation COURS

id_cours	intitule	nb_heures
C1	Mathématiques	40
C2	Informatique	60
C3	Physique	30

Relation INSCRIPTION

id_etudiant	id_cours	note
1001	C1	15
1001	C2	18
1002	C1	12
1003	C2	14

C. OPÉRATIONS UNAIRES

Les opérations unaires s'appliquent à une seule relation.

Types d'opérations unaires

— **Sélection** (σ) $\rightarrow$ Filtre horizontal (lignes)

— **Projection** (π) $\rightarrow$ Filtre vertical (colonnes)

— **Renommage** (ρ) $\rightarrow$ Modification des noms

I. Sélection (σ) - Restriction

Définition : La sélection extrait les tuples d'une relation qui satisfont une condition donnée. C'est un **filtre horizontal**.

Notation : $\sigma_{\text{condition}}(R)$

Propriétés :

— Le schéma reste identique

— La cardinalité diminue ou reste égale

— Opération de filtrage des lignes

Exemple 1 : Sélectionner les étudiants habitant à Paris

$$\sigma_{\text{ville='Paris'}}(\text{ÉTUDIANT})$$

Résultat :

id_etudiant	nom	prenom	ville
1001	Dupont	Marie	Paris
1003	Durand	Sophie	Paris

Conditions complexes :

- **ET logique** ($\wedge$) : $\sigma_{\text{ville='Paris'} \wedge \text{nom='Dupont'}}(\text{ÉTUDIANT})$
- **OU logique** ($\vee$) : $\sigma_{\text{ville='Paris'} \vee \text{ville='Lyon'}}(\text{ÉTUDIANT})$
- **Négation** ($\neg$) : $\sigma_{\neg(\text{ville='Paris'})}(\text{ÉTUDIANT})$
- **Comparaisons** : $\sigma_{\text{note}>15}(\text{INSCRIPTION})$

II. Projection (π)

Définition : La projection extrait certains attributs d'une relation en éliminant automatiquement les doublons. C'est un **filtre vertical**.

Notation : $\pi_{\text{liste_attributs}}(R)$

Propriétés :

- Le degré diminue ou reste égal
- Élimine les doublons automatiquement
- Opération de filtrage des colonnes

Exemple 1 : Extraire les noms et prénoms des étudiants

$$\pi_{\text{nom, prenom}}(\text{ÉTUDIANT})$$

Résultat :

nom	prenom
Dupont	Marie
Martin	Pierre
Durand	Sophie
Bernard	Jean

Exemple 2 : Extraire les villes (avec élimination des doublons)

$$\pi_{\text{ville}}(\text{ÉTUDIANT})$$

Résultat :

ville
Paris
Lyon
Marseille

III. Renommage (ρ)

Définition : Le renommage permet de modifier le nom d'une relation ou de ses attributs.

Notations :

- Renommer la relation : $\rho_S(R)$
- Renommer un attribut : $\rho_{B/A}(R)$ (remplace A par B)
- Renommer relation et attributs : $\rho_{S(B_1,B_2,\ldots,B_n)}(R)$

Utilité : Éviter les conflits de noms lors des jointures, clarifier la sémantique.

D. COMPOSITION D'OPÉRATIONS UNAIRES

Les opérations peuvent être combinées pour former des requêtes complexes.
Exemple : Extraire les noms des étudiants habitant à Paris

$$\pi_{\text{nom}}(\sigma_{\text{ville='Paris'}}(\text{ÉTUDIANT}))$$

Évaluation étape par étape :

1. **Sélection :** $\sigma_{\text{ville='Paris'}}(\text{ÉTUDIANT}) \rightarrow$ Filtre les étudiants parisiens
2. **Projection :** $\pi_{\text{nom}} \rightarrow$ Extrait uniquement les noms

Résultat final :

nom
Dupont
Durand

E. OPÉRATIONS ENSEMBLISTES

Ces opérations traitent les relations comme des ensembles mathématiques. Elles nécessitent que les relations soient **compatibles** (même nombre d'attributs, domaines compatibles).

I. Union ($\cup$)

Définition : L'union de deux relations R et S contient tous les tuples présents dans R ou dans S, sans doublons.
Notation : $R \cup S$
Condition de compatibilité :

— Même nombre d'attributs
— Domaines compatibles pour chaque attribut

II. Intersection ($\cap$)

Définition : L'intersection contient les tuples présents **simultanément** dans les deux relations.
Notation : $R \cap S$

III. Différence ($-$)

Définition : La différence $R - S$ contient les tuples présents dans R mais **pas** dans S.
Notation : $R - S$
Propriété importante : L'opération n'est **pas commutative** : $R - S \neq S - R$

F. PRODUIT CARTÉSIEN ($\times$)

Définition : Le produit cartésien de deux relations R et S produit une nouvelle relation contenant **toutes les combinaisons possibles** des tuples de R avec les tuples de S.
Notation : $R \times S$
Propriétés :

— Si R a n tuples et S a m tuples, alors $R \times S$ a $n \times m$ tuples
— Le degré de $R \times S$ est la somme des degrés de R et S

Utilisation : Le produit cartésien seul génère souvent des résultats non pertinents. Il sert de base théorique aux jointures.

G. JOINTURES

La jointure est l'opération la plus importante en pratique. Elle permet de combiner des informations provenant de plusieurs relations en utilisant des valeurs communes.

I. Jointure conditionnelle (θ-join)

Définition : Jointure de deux relations selon une condition θ quelconque.
Notation : $R \bowtie_\theta S$
Équivalence : $R \bowtie_\theta S = \sigma_\theta(R \times S)$

II. Équi-jointure

Définition : Cas particulier où la condition θ est une égalité.
Exemple : Associer chaque inscription à l'étudiant correspondant

$$\text{ÉTUDIANT} \bowtie_{\text{ÉTUDIANT.id_etudiant} = \text{INSCRIPTION.id_etudiant}} \text{INSCRIPTION}$$

III. Jointure naturelle ($\bowtie$)

Définition : Jointure équi sur les attributs **de même nom**, en éliminant les colonnes dupliquées.
Notation : $R \bowtie S$

IV. Jointures externes

Définition : Conservent les tuples qui n'ont pas de correspondance dans l'autre relation.
Types :

- **Jointure externe gauche** : conserve tous les tuples de la relation gauche
- **Jointure externe droite** : conserve tous les tuples de la relation droite
- **Jointure externe complète** : conserve tous les tuples des deux relations

H. DIVISION ($\div$)

Définition : La division répond aux requêtes de type "tous les X qui sont en relation avec **tous** les Y".
Notation : $R \div S$
Condition : Les attributs de S doivent être inclus dans ceux de R.
Exemple : Trouver les étudiants inscrits à TOUS les cours

$$\text{INSCRIPTION} \div \text{COURS}$$

I. EXEMPLE COMPLET DE REQUÊTE COMPLEXE

Question : Trouver les noms des étudiants parisiens ayant une note supérieure à 15.
Décomposition étape par étape :

1. Sélectionner les étudiants parisiens :

$$R_1 = \sigma_{\text{ville='Paris'}}(\text{ÉTUDIANT})$$

2. Sélectionner les inscriptions avec note > 15 :

$$R_2 = \sigma_{\text{note}>15}(\text{INSCRIPTION})$$

3. Joindre les deux résultats :

$$R_3 = R_1 \bowtie_{\text{id_etudiant}} R_2$$

4. Projeter sur les noms :

$$R_4 = \pi_{\text{nom}}(R_3)$$

Expression complète :

$$\pi_{\text{nom}}\left(\sigma_{\text{ville='Paris'}}(\text{ÉTUDIANT}) \bowtie_{\text{id_etudiant}} \sigma_{\text{note}>15}(\text{INSCRIPTION})\right)$$

J. PROPRIÉTÉS DES OPÉRATIONS

I. Commutativité

Opérations commutatives :

$$R \cup S = S \cup R \tag{1}$$
$$R \cap S = S \cap R \tag{2}$$
$$R \times S = S \times R \text{ (à l'ordre des attributs près)} \tag{3}$$
$$R \bowtie S = S \bowtie R \tag{4}$$

Opérations non commutatives :

$$R - S \neq S - R \tag{5}$$
$$R \div S \neq S \div R \tag{6}$$

II. Associativité

$$R \cup (S \cup T) = (R \cup S) \cup T \tag{7}$$
$$R \cap (S \cap T) = (R \cap S) \cap T \tag{8}$$
$$R \bowtie (S \bowtie T) = (R \bowtie S) \bowtie T \tag{9}$$

III. Distributivité

La sélection est distributive sur l'union :

$$\sigma_\theta(R \cup S) = \sigma_\theta(R) \cup \sigma_\theta(S)$$

La projection est distributive sur l'union :

$$\pi_A(R \cup S) = \pi_A(R) \cup \pi_A(S)$$

K. SYNTHÈSE DES OPÉRATIONS

Synthèse des opérations

OPÉRATIONS UNAIRES :

- Sélection (σ) → Filtre les lignes
- Projection (π) → Sélectionne les colonnes
- Renommage (ρ) → Renomme relation/attributs

OPÉRATIONS BINAIRES :

- Union ($\cup$) → Combine sans doublons
- Intersection ($\cap$) → Éléments communs
- Différence ($-$) → Éléments de R1 pas dans R2
- Produit cartésien ($\times$) → Toutes les combinaisons
- Jointure ($\bowtie$) → Combine selon une condition
- Division ($\div$) → Requêtes "pour tous"

Correspondance avec SQL

Algèbre Relationnelle	SQL
Sélection (σ)	`WHERE`
Projection (π)	`SELECT`
Union ($\cup$)	`UNION`
Intersection ($\cap$)	`INTERSECT`
Différence ($-$)	`EXCEPT` / `MINUS`
Produit cartésien ($\times$)	`FROM table1, table2`
Jointure ($\bowtie$)	`JOIN ... ON`
Division ($\div$)	`NOT EXISTS` (imbriqués)

L. RÈGLES D'OPTIMISATION

I. Principes généraux

Règle 1 : Effectuer les sélections le plus tôt possible pour réduire la taille des relations intermédiaires.

Règle 2 : Effectuer les projections tôt pour réduire le nombre d'attributs.

Règle 3 : Remplacer les produits cartésiens suivis de sélections par des jointures.

Règle 4 : Regrouper les opérations de même type.

II. Exemple d'optimisation

Requête non optimisée :

$$\sigma_{\text{ville='Paris'}}(\text{ÉTUDIANT} \times \text{INSCRIPTION})$$

Requête optimisée :

$$\sigma_{\text{ville='Paris'}}(\text{ÉTUDIANT}) \bowtie \text{INSCRIPTION}$$

Avantage : Réduction significative du nombre de tuples avant la jointure.

M. CONCLUSION

L'algèbre relationnelle constitue le socle théorique indispensable à la compréhension et à la maîtrise des bases de données relationnelles. Elle fournit :

- Un **cadre formel** pour la manipulation des données
- Les **fondements mathématiques** du langage SQL
- Les **bases de l'optimisation** des requêtes
- Une **approche systématique** pour résoudre des problèmes complexes

Points clés à retenir :

- Les huit opérations fondamentales permettent d'exprimer toute requête
- La propriété de fermeture autorise la composition d'opérations
- L'optimisation repose sur les propriétés mathématiques des opérations
- Chaque opération d'algèbre relationnelle a son équivalent en SQL

La maîtrise de l'algèbre relationnelle est essentielle pour :

- Comprendre le fonctionnement interne des SGBD
- Écrire des requêtes SQL efficaces
- Optimiser les performances des bases de données
- Concevoir des algorithmes de traitement de données

Le chapitre suivant abordera l'implémentation pratique de ces concepts à travers le langage SQL : syntaxe, requêtes avancées et optimisation.